Internship Daily Documentation

Day 4 — Thursday, April 30, 2026

Smart Internship & Hiring Portal | Intern: Mageswaran G

Field	Details
Date	Thursday, April 30, 2026
Intern	Mageswaran G
Project	Smart Internship & Hiring Portal
Mentor	Sandiya
Module	Week 1 — Module 1 (Final Day)
Day Goal	Complete frontend auth UI and connect to backend. Finalize Module 1.

1. Start of Day (SOD) — Morning Plan

SOD mail sent to Sandiya in the morning. Plan was to complete frontend authentication and finalize Module 1.

Planned Tasks:

- Set up React frontend project using Vite + Tailwind CSS
- Establish project structure following feature-based architecture
- Build Login UI component with proper form handling
- Build Signup UI component aligned with backend validation rules
- Integrate frontend with backend authentication APIs (signup/login)
- Implement token handling — access token in memory, refresh in HTTP-only cookie
- Configure routing and protected route setup for authenticated access

2. What Was Built Today

2.1 React + Vite + Tailwind CSS Setup

- Created React project using Vite (npm create vite@latest frontend -- --template react)
- Installed dependencies: axios, react-router-dom
- Installed Tailwind v4 with @tailwindcss/vite plugin
- Configured vite.config.js with tailwindcss() plugin
- Updated src/index.css with single line: @import 'tailwindcss'
- Tested: Browser showed 'Tailwind is working!' in blue bold text

2.2 Frontend Folder Structure

Created 3 new folders inside src/:

- src/context/ — for AuthContext.jsx (shared memory)
- src/pages/ — for LoginPage.jsx and SignupPage.jsx
- src/services/ — for authService.js (API calls)

2.3 authService.js

File: frontend/src/services/authService.js

- Created axios instance with baseURL pointing to backend (http://127.0.0.1:8000/api/v1)
- Added withCredentials: true to allow cookies (refresh token)
- Added axios interceptor — reads token from window.__accessToken__ and attaches to Authorization header automatically
- signup() function — calls POST /auth/signup
- login() function — calls POST /auth/login

2.4 AuthContext.jsx

File: frontend/src/context/AuthContext.jsx

- Created shared memory using React Context API
- Stores accessToken in React state (memory — NOT localStorage, safe from XSS)
- Stores user object {id, name, email, role}
- loginUser() — saves token to state AND window.__accessToken__ (for axios)
- logoutUser() — clears both state and window token
- useAuth() shortcut hook exported for easy use in all pages

2.5 SignupPage.jsx

File: frontend/src/pages/SignupPage.jsx

- Form with 4 fields: Full Name, Email, Password, Role (dropdown)
- handleChange() — updates formData state on every keystroke
- handleSubmit() — calls signup API, shows success/error message
- Replaced alert() with green UI success box (Fix 1)
- setTimeout 2 seconds then navigates to /login automatically
- Red error box shows backend error messages
- Loading state: button shows 'Creating Account...' while API runs

2.6 LoginPage.jsx

File: frontend/src/pages/LoginPage.jsx

- Form with 2 fields: Email and Password
- Calls login API via authService
- On success: saves token to AuthContext memory, navigates to /dashboard
- Error handling: shows backend errors in red box
- Loading state: button shows 'Logging in...' while API runs

2.7 App.jsx — Routing

File: frontend/src/App.jsx

- BrowserRouter wraps entire app for navigation
- AuthProvider wraps everything so all pages share token
- Route / → redirects to /login
- Route /signup → SignupPage
- Route /login → LoginPage
- Route /dashboard → ProtectedRoute wrapping Dashboard
- ProtectedRoute: if token exists show page, else redirect to /login
- Dashboard: shows Welcome message with user name, email, role + Logout button

2.8 HirePortal Branding

- Updated frontend/index.html with professional title: HirePortal — Smart Internship & Jobs
- Added meta description for SEO
- Created custom orange+teal logo on favicon.io using Montserrat font
- Downloaded logo.png and logo.svg, placed in frontend/public/
- Browser tab now shows HirePortal name with orange HP logo

3. Fixes Applied Today

Fix	Problem	Solution
Fix 1	alert() popup — ugly and blocks UI	Replaced with green success box with 2-second auto-redirect
Fix 2	Token not sent to backend APIs	Added axios interceptor to auto-attach Bearer token from window.__accessToken__
Fix 3	userController used old Day 2 pattern	Refactored to use userService + ApiResponse standard format
Fix 4	refreshToken visible in GET /users response	Added .select('-password -refreshToken') in userService
Fix 5	Zod bug in errorMiddleware: err.errors	Fixed to err.issues (Zod v4 standard)
Fix 6	Duplicate getUserById in userService	Removed duplicate — kept version with .select() for security
Fix 7	CORS error: port mismatch	Updated CLIENT_URL in .env to match frontend port (5173)
Fix 8	Duplicate loginUser in AuthContext	Cleaned file — single loginUser function with window token

4. Testing Done

Test	URL/Action	Result

Signup — new user	POST /api/v1/auth/signup	201 — Green success box shows, redirects to login
Signup — duplicate email	Same email again	400 — Red error box: Email already registered
Signup — weak password	Password < 8 chars	400 — Validation error shown
Login — correct credentials	POST /api/v1/auth/login	200 — Dashboard shows with name and role
Login — wrong password	Wrong password	401 — Red error: Invalid credentials
Dashboard access — logged in	localhost:5173/dashboard	Dashboard shows correctly
Dashboard access — not logged in	Direct URL without login	Redirected to /login automatically
Logout	Click Logout button	Token cleared, redirected to login
GET /users — security	Thunder Client GET	No password, no refreshToken in response
GET /users profile	GET /api/v1/users/profile	Returns user data without sensitive fields

5. Mistakes Made & How Fixed

#	Mistake	Root Cause	Fix Applied
1	Signup failed — CORS error	CLIENT_URL in .env was http://127.0.0.1:3000, frontend on 5174	Changed CLIENT_URL to match actual frontend port
2	Frontend showing on port 5174 instead of 5173	Old server already using 5173	Added port: 5173, strictPort: true to vite.config.js
3	loginUser declared twice in AuthContext	Copy-paste when adding window token	Deleted duplicate, kept single clean function
4	getUserById duplicated in userService	Added new version but forgot to remove old one	Removed old version without .select()
5	ChatGPT suggested localStorage for JWT	ChatGPT did not know our security architecture	Rejected — kept memory storage (window.__accessToken__)
6	ChatGPT suggested @tailwind directives	ChatGPT confused Tailwind v3 and v4	Rejected — kept @import 'tailwindcss' (v4 correct way)
7	JOBS image from Google for favicon	Downloaded copyright image	Rejected — created own logo on favicon.io (free, safe)

6. Key Concepts Learned Today

React useState

Stores data that can change in a component. When state changes, the component re-renders automatically.

const [value, setValue] = useState(initialValue)

React Context API

Shared memory for entire app. Any page can read from it without passing props.

Used for: storing JWT token and user info after login.

Axios Interceptor

Runs automatically before every API request. Attaches JWT token to Authorization header.

So every page never needs to manually add the token.

Protected Routes

A wrapper component that checks if user is logged in.

If token exists: show the page. If no token: redirect to login.

CORS — Cross Origin Resource Sharing

Backend security rule — only accepts requests from trusted frontend URL.

CLIENT_URL in .env must exactly match the frontend URL including port number.

window.__accessToken__

Global browser memory. Available to all JavaScript code. Not saved after refresh.

Safer than localStorage (XSS cannot steal it) and accessible by axios interceptor.

Tailwind v4 vs v3

Feature	Tailwind v3 (old)	Tailwind v4 (new — what we use)
CSS import	@tailwind base; @tailwind components; @tailwind utilities;	@import 'tailwindcss'; (one line)
Config file	tailwind.config.js required	Not needed
Vite plugin	Not available	@tailwindcss/vite plugin

7. GitHub Commits Today

Commit Message	What Changed
Week 1 complete: Full auth system - backend + frontend	20 files — all frontend files added
Day 4: Logo, alert fix, axios interceptor, userController refactor	10 files — fixes and improvements
Day 4: Fix duplicate loginUser in AuthContext	AuthContext cleaned

Day 4: Fix duplicate getUserById and Zod issues bug	userService and errorMiddleware fixed

8. EOD (End of Day) — Mail Sent to Sandiya

Mail sent in the evening with complete status update. All planned tasks completed successfully.

9. Tomorrow — Friday Weekly Review

Meeting with Sandiya for Week 1 progress review.

What to demonstrate:

- Open http://localhost:5173
- Show Signup form — fill details — green success message
- Show Login — dashboard with real name and role
- Show protected route — direct URL redirects to login
- Show GitHub — all commits from Day 1 to Day 4

Next Plan — Module 2 (Week 2):

- Candidate profile creation — name, skills, bio, location
- Company profile creation
- Resume upload using Multer
- Profile edit functionality

10. Important Notes for Future Reference

- Always create .gitignore BEFORE first commit — never push .env or node_modules
- CORS: CLIENT_URL must exactly match frontend port — even one difference blocks all requests
- Never store JWT in localStorage — use React memory + window for secure storage
- Tailwind v4 uses @import 'tailwindcss' — not the old @tailwind directives
- Axios interceptor reads token from window.__accessToken__ — set after login, cleared after logout
- ProtectedRoute pattern: accessToken ? children : <Navigate to='/login' />
- Always run backend first then frontend — both must be running for app to work
- Port 5173 locked using strictPort: true in vite.config.js to avoid CORS mismatch
- ChatGPT suggestions must be verified against your actual code before applying
- Working code must never be changed just because AI says it is wrong — test first